

Spike: Task 12**Title:** Command Pattern**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to demonstrate my ability to collect and analyse software performance data while applying my knowledge in comparing and evaluating functions, IDE and compiler settings.

Besides this report, what else was created?

- Code files in see C:\Users\storm\Documents\GitHub\GAMES PROGRAMMING REPO\COS30031-2023-102608927\12 - Spike – Command Pattern\Zorkish\Zorkish

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Game Specification Details Canvas Module
- OO Concepts, Patterns & Structures Canvas Modules
 - 2.1 Software Architecture of Games Lecture Slides
 - 2.2 Game States and Stages Lecture Slides
 - 3.2 Data Structures
 - 3.3 Design Patterns

Tasks undertaken:

- Use Task 8 Zorkish Code
- Planning How to implement Command Processing

Planning Phase:

Using Phase 1 Task 8 code and building upon it to implement command processing and file reading with the help of other previous tasks. I decided to use the JSON library to create a JSON file to read from for location and create a command class to create sub command classes.

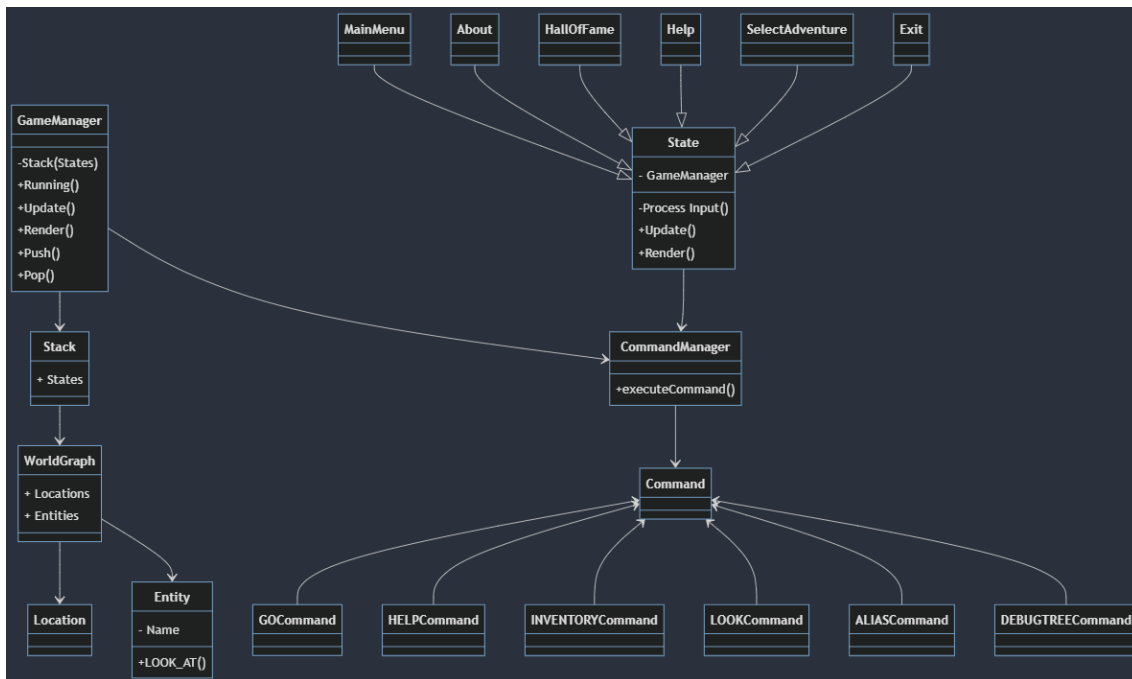


Figure 1 : Updated UML Design for Phase 2 of Zorkish Adventure Game

Implementing GameWorld:

```

#include "SelectAdventure.h"

SelectAdventure::SelectAdventure(GameManager* manager)
{
    _manager = manager;
}

SelectAdventure::~SelectAdventure()
{
}

void SelectAdventure::update()
{
    processInput();

    switch (_command)
    {
        case '1':
            _manager->pop_state();
            _manager->push_state(new Gameplay(_manager, "world1.json"));
            break;
        case '2':
            _manager->pop_state();
            _manager->push_state(new Gameplay(_manager, "world1.json"));
            break;
        case '3':
            _manager->pop_state();
            _manager->push_state(new Gameplay(_manager, "world1.json"));
            break;
        default:
            cout << "Invalid input." << endl;
    }
}
  
```

We modified **SelectAdventure** to be able to read from our .JSON file that consists of data regarding our locations with connections and items stored in them.

```
#include "GameWorld.h"

GameWorld::GameWorld(json data) : _running(true)
{
    for (json loc : data["locations"])
    {
        _locations.push_back(new Location(loc));
    }

    Inventory* inv = new Inventory();

    _player = new Player(_locations.front(), inv);
    _commandManager = new CommandManager(_player);

    cout << "Welcome to Zorkish Adventure" << endl;
}

GameWorld::~GameWorld() {}

Location* GameWorld::getLocation(string LocName)
{
    for (Location* loc : _locations)
    {
        if (loc->getName() == LocName)
            return loc;
    }

    return NULL;
}

vector<string> GameWorld::splitString(string input)
{
    vector<string> result;

    size_t pos = 0;
    string token;
    string delimiter = " ";
    while ((pos = input.find(delimiter)) != string::npos) {
        token = input.substr(0, pos);
        result.push_back(token);
        input.erase(0, pos + delimiter.length());
    }
    result.push_back(input);

    return result;
}

vector<string> GameWorld::processInput()
{
    string input;
    vector<string> commands;

    getline(cin, input);

    return splitString(input);
}

void GameWorld::movePlayer()
{
    _player->setLocation(getLocation(_player->getLocName()));
}

bool GameWorld::running()
{
    return _running;
}

void GameWorld::update()
{
    _commandManager->processCommand(processInput());
    movePlayer();
}

void GameWorld::render()
{
    cout << _player->getLocation()->getDesc() << endl << ">> ";
}
```

We use GameWorld to act as a central hub for our game's world which handles player's inventory, location, movement, player input, updating and rendering regarding game states. GameWorld is initialised by using our JSON file data where it creates and store's locations, initialises player inventory and starting location and most importantly handles player commands.

Implementing Command Processing

```

1  #pragma once
2  #include <iostream>
3  #include <vector>
4  #include "Player.h"
5
6  using namespace std;
7
8  class Command
9  {
10 protected:
11
12 public:
13     virtual void execute(vector<string>, Player*) = 0;
14 };

```

```

1  #include "CommandManager.h"
2
3  CommandManager::CommandManager(Player* player) : _player(player)
4  {
5      _commands.emplace("go", new GoCommand());
6      _commands.emplace("look", new LookCommand());
7      _commands.emplace("help", new HelpCommand(this));
8      _commands.emplace("inventory", new InventoryCommand());
9      _commands.emplace("alias", new AliasCommand(this));
10     _commands.emplace("debug", new DebugCommand());
11 }
12
13 CommandManager::~CommandManager()
14 {
15 }
16
17 void CommandManager::processCommand(vector<string> command)
18 {
19     if (_commands.find(command.at(0)) != _commands.end())
20     {
21         _commands.at(command.at(0))->execute(command, _player);
22     }
23     else
24         cout << "Invalid command." << endl;
25 }
26
27 void CommandManager::addCommand(string aliasName, string commandName)
28 {
29     for (auto existing : _commands)
30     {
31         if (existing.first == commandName)
32         {
33             _commands.emplace(aliasName, existing.second);
34             return;
35         }
36     }
37     cout << "Unable to add Command Alias." << endl;
38 }
39
40
41 void CommandManager::showCommands()
42 {
43     cout << "Available Commands: " << endl;
44     for (auto command : _commands)
45     {
46         cout << command.first << endl;
47     }
48 }
49
50

```

Using a Command base class and Command Manager we encapsulate commands to invoke them when a user input for a command has been received. In the processCommand constructor we use a set list of commands to be created and some commands need the use of pointers to reference it. Therefore, processing commands require inputting of a vector string to check against the commands dictionary and then execute it. We also store commandmanager in the GameWorld so that its called once per update.

```
1  #include "GoCommand.h"
2
3  void GoCommand::execute(vector<string> command, Player* player)
4  {
5      if (command.size() == 1)
6      {
7          cout << "No direction specified. Please try again and provide a direction." << endl;
8      }
9      else if (command.size() == 2)
10     {
11         Location* pLoc = player->getLocation();
12
13         const string pCon = pLoc->findConnection(command.at(1));
14
15         if (pCon == "")
16         {
17             cout << "Invalid direction. Please choose another direction." << endl;
18         }
19         else
20         {
21             player->setLocName(pCon);
22         }
23     }
24     else
25     {
26         cout << "Unable to process command. Please input command and single direction." << endl;
27     }
28 }
29
```

Using command class, we were able to create the command such as Go, Look, Alias, Debug, and Inventory commands. For example, Go Command is used to check if player can move to one of the locations connections / directions from their current location.

```
1  #include "DebugCommand.h"
2
3  void DebugCommand::execute(vector<string> command, Player* player)
4  {
5      cout
6          << "===DEBUG INFORMATION===" << endl
7          << "---PLAYER INFO---" << endl
8          << "Player Health: " << player->getHealth() << endl
9          << "Items Held: " << endl;
10     player->getInventory()->view();
11     cout
12         << "---LOCATION INFORMATION---" << endl
13         << "Location Name: " << player->getLocation()->getName() << endl
14         << "Location Desc: " << player->getLocation()->getDesc() << endl
15         << "Location Connections: " << endl;
16     player->getLocation()->showConnections();
17 }
18
```

Whereas Debug command which is a bit more complex outputs player information with the use of Player.cpp to get player's inventory and current location. Basically used to show useful debugging information regarding our game graph world and its entities.